Analytical FK/IK and Motion Planning for the ABB IRB 120

Ame Merrifield
ME 7751 – Robotics
Spring 2026

***Abstract*—This paper presents a complete kinematics and motion-planning implementation for the ABB IRB 120 robot. The project includes a forward kinematics (FK) model from Denavit–Hartenberg (DH) parameters, an analytical inverse kinematics (IK) solver with branch handling and singularity treatment, and a web-based motion-planning interface. Validation is performed using FK→IK→FK round-trip testing and randomized pose checks. In the web validation harness, 300/300 random targets were solved successfully. The resulting system is reproducible in both MATLAB and TypeScript and supports interactive visualization and trajectory generation.**

***Index Terms*—robotics, inverse kinematics, forward kinematics, ABB IRB 120, motion planning**

I. Introduction

This project addresses analytical modeling and implementation of six-degree-of-freedom robot kinematics for the ABB IRB 120. The core objective is to build an end-to-end pipeline: (1) FK for pose generation from joint coordinates, (2) IK for recovering joint candidates from target poses, and (3) a motion-planning application for interactive trajectory execution. The chosen platform is the ABB IRB 120 because it is an industrial manipulator with a spherical wrist structure that permits geometric decoupling in IK.

The implementation is split across two environments. MATLAB is used for derivation-oriented development and numerical diagnostics, while a React+TypeScript web app provides visualization, path playback, and user interaction. This dual implementation supports both analytical transparency and practical usability.

A. Robot Specifications and Joint Limits

The project robot is the ABB IRB 120-3/0.6 (6-DOF articulated arm). The joint limits used by both the MATLAB and web implementations are listed in Table I.

TABLE I
Joint Axis Limits (ABB IRB 120-3/0.6)

Joint	Low (deg)	High (deg)
θ_1	-165	165
θ_2	-110	110
θ_3	-90	70
θ_4	-160	160
θ_5	-120	120
θ_6	-400	400

B. Home Configuration

The home configuration used in this project is

$$q_{home} = [0,\ 0,\ 0,\ 0,\ 0,\ 0]^\circ. \tag{1}$$

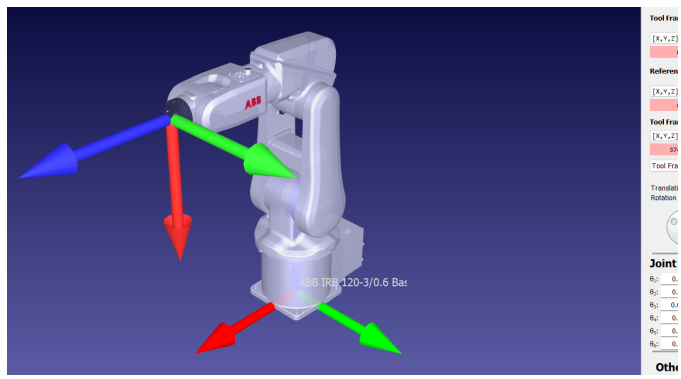

Fig. 1. ABB IRB 120 home configuration in RoboDK.

C. Important Robot Specs

Key ABB IRB 120-3/0.6 datasheet values used for planning assumptions and feasibility checks are summarized in Table II.

TABLE II
Important ABB IRB 120-3/0.6 Specifications

Specification	Value
Number of axes	6
Horizontal reach	0.58 m
Handling capacity	3 kg (4 kg with vertical wrist)
Position repeatability	0.01 mm
Robot weight	25 kg
Protection class	IP30
Mounting	Any angle
Controller	IRC5 Compact / IRC5 Single Cabinet
Rated transformer power	3.0 kVA
Power consumption	0.24 kW
Integrated wrist signals	10
Integrated wrist air supply	4 lines at 5 bar

II. Forward Kinematics

Using shorthand $c_{\theta i} = \cos\theta_i$, $s_{\theta i} = \sin\theta_i$, $c_{\alpha i} = \cos\alpha_i$, and $s_{\alpha i} = \sin\alpha_i$, the standard DH transform for each joint is

$$^{i-1}T_i = \begin{bmatrix} c_{\theta i} & -s_{\theta i}c_{\alpha i} & s_{\theta i}s_{\alpha i} & a_i c_{\theta i} \\ s_{\theta i} & c_{\theta i}c_{\alpha i} & -c_{\theta i}s_{\alpha i} & a_i s_{\theta i} \\ 0 & s_{\alpha i} & c_{\alpha i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

The end-effector transform is

$$^{0}T_6 = \prod_{i=1}^{6} {}^{i-1}T_i. \tag{3}$$

The DH parameters used in this project are listed in Table III.

TABLE III
ABB IRB 120 DH Parameters (Project Model)

i	α_i (deg)	a_i (mm)	d_i (mm)
1	-90	0	290
2	0	270	0
3	-90	70	0
4	90	0	302
5	-90	0	0
6	0	0	72

Validation was performed by checking expected home and nominal poses and by using FK-generated target transforms for IK regression tests. The MATLAB FK utility returns both per-link transforms and cumulative transforms, enabling frame-by-frame debugging and consistency checks.

A. FK Verification Against RoboDK

To verify the FK implementation against an external simulator, six representative joint configurations were selected across the robot workspace and joint-limit envelope. For each case, the end-effector pose from the project FK solver is compared with the pose reported by RoboDK (same robot model, same tool/frame convention).

The final manuscript reports position error as

$$\Delta p = \|p_{FK} - p_{RoboDK}\|_2 \tag{4}$$

and orientation error as the equivalent angle (degrees) between rotation matrices. Because RoboDK Euler values can appear in an equivalent branch, R_{RoboDK} is constructed from the equivalent representation before error evaluation:

$$\Delta R = \cos^{-1}\left(\frac{\mathrm{tr}(R_{FK}^T R_{RoboDK}) - 1}{2}\right). \tag{5}$$

III. Inverse Kinematics

The IK solver uses position–orientation decoupling. With target transform

$$T_{06} = \begin{bmatrix} R_{06} & p_{06} \\ 0 & 1 \end{bmatrix}, \tag{6}$$

the wrist center is

$$p_w = p_{06} - d_6 R_{06}(:,3). \tag{7}$$

Let $p_w = [p_x\ p_y\ p_z]^T$, $r = \sqrt{p_x^2 + p_y^2}$, and $s = p_z - d_1$. Shoulder branches are

$$q_{1a} = \mathrm{atan2}(p_y, p_x), \quad q_{1b} = q_{1a} + \pi. \tag{8}$$

For the arm triangle,

$$L_2 = a_2, \quad L_3 = \sqrt{a_3^2 + d_4^2}, \quad D = \frac{r^2 + s^2 - L_2^2 - L_3^2}{2L_2L_3}. \tag{9}$$

If $|D| > 1$, the target is unreachable. Elbow branches are

$$q_3' = \mathrm{atan2}(\pm\sqrt{1 - D^2}, D). \tag{10}$$

Then

$$q_2' = -(\phi + \psi), \tag{11}$$
$$\phi = \mathrm{atan2}(s, r), \tag{12}$$
$$\psi = \mathrm{atan2}(L_3 \sin q_3',\ L_2 + L_3 \cos q_3')\,. \tag{13}$$

Joint offsets are applied to recover robot joint variables.

For wrist orientation, the solver computes

$$R_{36} = R_{03}^T R_{06}, \tag{14}$$

and extracts (q_4, q_5, q_6) with wrist-flip branches. Singular behavior near $\sin(q_5) \approx 0$ is handled with a constrained fallback branch. Nominally, the solver yields up to eight candidates (2 shoulder $\times$ 2 elbow $\times$ 2 wrist).

IV. IK Verification

IK verification follows a systematic forward-then-inverse workflow:

1) Choose test joint configurations q_{test}, including near-singular and near-limit cases.
2) Compute target pose: $T_{test} = \mathrm{FK}(q_{test})$.
3) Apply IK to obtain all branches $q_{IK}^{(k)}$.
4) Verify each branch using reconstruction $\mathrm{FK}(q_{IK}^{(k)}) \approx T_{test}$.
5) Cross-validate selected poses in RoboDK and compare against analytical IK branches.

Table V reports the deterministic test set used here. Cases C5–C8 intentionally stress near-limit and near-singular conditions.

Web validation reports 300/300 solved random targets (100% success). MATLAB scripts (`run_20_tests`, `verify_ik_correct`) were used for additional numerical diagnostics.

These results show that direct angle-by-angle branch matching is only partially reliable across solvers, especially at wrist singularities where multiple (q_4, q_6) decompositions produce the same orientation. Therefore, pose-level consistency (Δp, ΔR) is used as the primary cross-validation criterion against RoboDK.

TABLE IV
FORWARD KINEMATICS VERIFICATION CASES (PROJECT FK VS ROBODK)

Case	Joint vector $[q_1, \ldots, q_6]$ (deg)	FK pose (x, y, z, rz, ry, rx)	RoboDK pose (x, y, z, rz, ry, rx)	$(\Delta p, \Delta R)$
T1	[0, 0, 0, 0, 0, 0]	(374.000, 0.000, 630.000, -180.000, -90.000, 0.000)	(374.000, 0.000, 630.000, 0.000, 90.000, 0.000)	(0.0000, 0.0000)
T2	[30, -45, 20, 40, -30, 60]	(96.492, 28.990, 723.337, -67.949, 18.032, -41.412)	(96.492, 28.990, 723.337, 112.051, -18.032, 41.412)	(0.0006, 0.0006)
T3	[-60, 35, -40, -70, 45, -120]	(209.494, -458.536, 594.316, 127.581, 74.149, -131.030)	(209.494, -458.536, 594.316, -52.419, -74.149, 131.030)	(0.0007, 0.0005)
T4	[90, -80, 50, 100, -60, 200]	(61.407, -2.768, 557.130, 129.141, -50.287, 79.197)	(61.407, -2.768, 557.130, -50.859, 50.287, -79.197)	(0.0006, 0.0008)
T5	[-120, 70, -70, 140, 80, -300]	(-244.638, -514.881, 506.663, -69.021, 38.549, -15.287)	(-244.638, -514.881, 506.663, 110.979, -38.549, 15.287)	(0.0007, 0.0007)
T6	[150, -100, 65, -150, -100, 350]	(11.445, -47.546, 416.203, -14.127, 30.404, 157.744)	(11.445, -47.546, 416.203, 165.873, -30.404, -157.744)	(0.0005, 0.0007)

TABLE V
SYSTEMATIC IK VERIFICATION CASES (FK→IK→FK)

Case	q_{test} (deg)	T_{test} from FK (x, y, z, rz, ry, rx) (mm,deg)	IK branches N	Best Δp (mm)	Best rotational error
C1	[0, 0, 0, 0, 0, 0]	(374.000, 0.000, 630.000, -180.000, -90.000, 0.000)	6	0.000000	1.721e-16 (Fro)
C2	[30, -45, 20, 40, -30, 60]	(96.492, 28.990, 723.337, -67.949, 18.032, -41.412)	8	0.000000	0.000e+00 (Fro)
C3	[-60, 35, -40, -70, 45, -120]	(209.494, -458.536, 594.316, 127.581, 74.149, -131.030)	8	0.000000	0.000e+00 (Fro)
C4	[150, -100, 65, -150, -100, 350]	(11.445, -47.546, 416.203, -14.127, 30.404, 157.744)	8	0.000000	8.532e-16 (Fro)
C5	[0, -110, -90, 0, 0, 0]	(-581.221, 0.000, 3.961, 0.000, 70.000, -180.000)	8	0.000000	1.570e-16 (Fro)
C6	[0, 110, 70, 0, 0, 0]	(-120.283, 0.000, 127.655, 180.000, 90.000, 0.000)	8	0.000000	1.743e-32 (Fro)
C7	[45, 0, 0, 90, 0, -90]	(264.458, 264.458, 630.000, -135.000, -90.000, 0.000)	6	0.000000	1.688e-16 (Fro)
C8	[-120, 70, -70, 140, 80, -300]	(-244.638, -514.881, 506.663, -69.021, 38.549, -15.287)	8	0.000000	0.000e+00 (Fro)

TABLE VI
ROBODK CROSS-VALIDATION SUMMARY

Case	RoboDK IK branch used	Match vs analytical IK	Notes
C2	6 raw (3 unique wrapped)	4/6 direct joint matches (1° tol)	Pose-consistent; max $\Delta p \leq 0.374$ mm, $\Delta R \leq 0.090^\circ$.
C4	10 raw (4 unique wrapped)	5/10 direct joint matches (1° tol)	Pose-consistent; max $\Delta p \leq 0.027$ mm, $\Delta R \leq 0.062^\circ$.
C5	3 raw (1 unique wrapped)	0/3 direct joint matches	Wrist-singular ($q_5 \approx 0$); tuples differ but pose is equivalent ($\Delta p = 0$, $\Delta R = 0$).
C7	4 raw (2 unique wrapped)	0/4 direct joint matches	Wrist-singular ($q_5 \approx 0$); alternate q_4/q_6 split, same pose; max $\Delta p \leq 0.200$ mm.

V. MOTION PLANNING APPLICATION

The motion-planning application is implemented as a React+TypeScript web app with a modular architecture:

- **Kinematics layer:** FK/IK functions and validation helpers.
- **Planning layer:** trajectory generation for joint-space interpolation and task-space targets.
- **UI layer:** robot viewer, controls, and charts for joint trajectories.

The implementation follows the interpolated-motion formulation in the lecture slides [1]. Using this report's notation, joint variables are written as q_i (instead of θ_i in some slide formulas). For joint interpolation,

$$\mathrm{moveJ}(t) = [q_1(t) \ \cdots \ q_n(t)]^T , \tag{15}$$

with endpoint joint vectors obtained from IK as

$$q_S = \mathrm{IK}(P_S), \qquad q_E = \mathrm{IK}(P_E). \tag{16}$$

For Cartesian interpolation, the app uses linear position plus angle-axis orientation interpolation,

$$\mathrm{moveL}(t) = [x(t) \ y(t) \ z(t) \ \phi(t)]^T , \tag{17}$$

with

$$\phi = \cos^{-1}\left(\frac{\mathrm{tr}(R) - 1}{2}\right), \qquad \mathbf{k} = \frac{1}{2\sin\phi}\begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}. \tag{18}$$

The interpolated orientation is computed using Rodrigues' formula in exponential-map form:

$$R(s) = R_0 \exp\big([\mathbf{k}]_\times \, \phi \, \sigma(s)\big), \qquad p(s) = p_0 + \sigma(s)(p_1 - p_0). \tag{19}$$

Time scaling uses either cubic or LSPB. Let $\Delta q = q_f - q_0$ and $s = t/t_f \in [0, 1]$. For cubic interpolation (the app's `smoothstep01`),

$$q(t) = q_0 + \Delta q \left(3s^2 - 2s^3\right), \tag{20}$$

$$\dot{q}(t) = \frac{\Delta q}{t_f}\left(6s - 6s^2\right), \qquad \ddot{q}(t) = \frac{\Delta q}{t_f^2}\left(6 - 12s\right). \tag{21}$$

For LSPB (parabolic blend), the app uses normalized blend fraction $r = t_b/t_f$ (clamped to $0.05 \le r \le 0.45$), which is equivalent to the lecture piecewise form:

$$\sigma(s) = \begin{cases} \dfrac{s^2}{2r(1-r)}, & 0 \le s < r, \\ \dfrac{s - r/2}{1-r}, & r \le s \le 1-r, \\ 1 - \dfrac{(1-s)^2}{2r(1-r)}, & 1-r < s \le 1. \end{cases} \tag{22}$$

Joint trajectories are then evaluated as

$$q(t) = q_0 + \Delta q \, \sigma\left(\frac{t}{t_f}\right). \tag{23}$$

This corresponds to a trapezoidal velocity profile with parabolic blends at the start and end, consistent with the lecture derivation and with the app's `lspbScale01` implementation.

For the representative UI demonstration, the sample joint-space waypoints were $q_{start} = [0, -20, 20, 0, 30, 0]^\circ$ and $q_{end} = [90, -80, 50, 100, -60, 200]^\circ$.

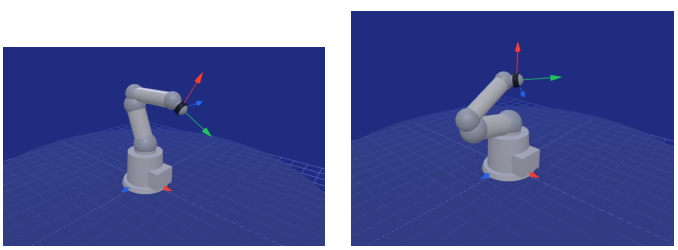

Fig. 2. Motion-planning app UI at the selected sample start and end joint positions.

MoveJ-style behavior is implemented via joint-space interpolation between IK-valid waypoints, while task-space commands use IK at sampled trajectory states. The GUI supports interactive target selection and visual verification of resulting motion, allowing fast comparison between candidate IK branches and planned execution paths.

VI. Conclusions

This project delivers a complete ABB IRB 120 kinematics pipeline from analytical derivation to interactive deployment. The FK and IK implementations are consistent across MATLAB and TypeScript, and randomized validation demonstrates strong reconstruction performance.

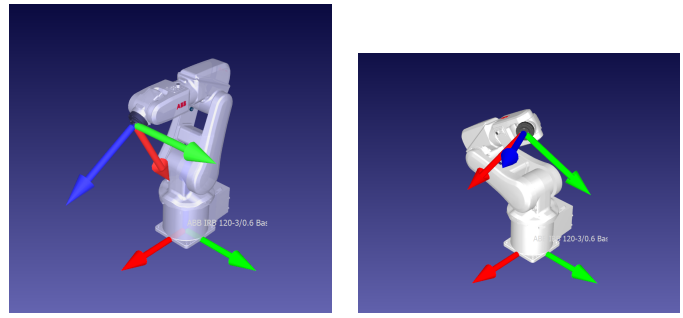

Fig. 3. RoboDK visualization at the same sample start and end joint positions.

Key lessons include the importance of DH-convention consistency, careful branch bookkeeping, and explicit singularity handling in wrist extraction. Current limitations include branch ranking under continuity constraints and behavior near degenerate wrist configurations. Future work will prioritize continuity-aware solution selection, stronger singularity regularization, and expanded stress tests near workspace boundaries.

References

[1] MECHENG 7751, "L4 Path Planning," lecture slides, The Ohio State University, 2026.

[2] ABB Robotics, "IRB 120 Product Specification," Rev. J, Nov. 2019.